

What to build, in what order, and what to prove first

A twelve-week plan sequenced by technical risk rather than by feature. The hardest unsolved problem in this system is offline signing without a backdating hole — so that gets built in week two, not week ten.

12 weeks 4 spikes before any code 3 records to capture 1 demo that must not fail

00 — THE PRINCIPLE

Build the risky thing first, while there is still time to be wrong

Most student projects build the easy, visible parts first — the dashboard, the map, the alert feed — and discover the hard problem in the final fortnight. Your hard problem is specific and known: **a device that signs offline can claim any timestamp it likes**. If that cannot be bounded, the evidence chain does not work, and you need to know that in week two.

So the order below is roughly the inverse of how impressive each piece looks. The dashboard is week nine. The cryptographic spine is week two.

01 — CRITICAL PATH

The dependency chart

WK 0

Second Schedule check
does s.3A cover our key?

StrongBox or TEE?
test the actual handset

Reach a TSA
RFC 3161, from India

Find
RPF

WK 1-3

Hash-chained append-only log
Merkle history tree · Postgres

Edge signing + attestation
Keystore/StrongBox · Verified Boot

Offline queue + TS
THE HARD ONE – bc

WK 4-6

Record 1 – s.43 trigger
what made the officer move

Record 2 – field test
reagent · lot · expiry · colour

Record 3 – RPF→
the seam · receipt

WK 7-9

s.63 certificate generator
Schedule Part A + B · SHA-256

Verifier + tamper demo
break the chain on stage

Minimal dashboard
case view · hand

WK 10-12

Field note · evidence pack · judge Q&A rehearsal
two weeks of real use at one station beats any architecture diagram

■ De-risking spike – answer before coding ■ Build ■ Proof & demo ■ Evidence & pitch

READ THE CHART THIS WAY

Each row depends on the row above it. The four red boxes are **questions, not code** – they take a day each and any one of them can change the design. Do them in week zero, before anyone writes a line.

02 – WEEK ZERO

Four questions to answer before writing code

SPIKE	THE QUESTION	HOW TO ANSWER IT	IF THE ANSWER IS BAD
Second Schedule	Does IT Act s.3A actually cover a self-generated device keypair? s.3A(1) reads "considered reliable; <i>and</i> may be specified in the Second Schedule" — conjunctive.	Read the Second Schedule and the CCA's approved-techniques list. Half a day.	You need a DSC from a CCA-licensed CA even for the prototype. Budget the paperwork now.
StrongBox	Does the handset you can actually afford have a discrete secure element, or only a TEE?	Write 30 lines of Kotlin, request a hardware-backed key, read the attestation certificate's security level.	TEE-only still works; your attestation claim is just weaker. Say so honestly rather than overclaiming.
Timestamp authority	Can you reach an RFC 3161 TSA reliably from India, and is there a second, independent one?	Curl a free TSA, time it. Then find a second in a different trust domain.	Fall back to publishing signed tree heads somewhere public. Never rely on one authority.
One real officer	Will one RPF or GRP officer talk to you for an hour about what actually happens after a seizure?	Ask. College liaison, a station house, anyone. One conversation.	Build from the Crime Manual and say the workflow is unvalidated. Weaker, still honest.

GATE 0 — DO NOT START WEEK 1 UNTIL

You can state, in one sentence each, what signature status you are building for, what your device can attest to, and how a record gets anchored to trusted time.

WK	BUILD	DONE WHEN	RISK RETIRED
0	The four spikes above	Four answers written down	DESIGN ASSUMPTIONS
1	Append-only log service. Postgres table, each row carrying the previous row's hash. Merkle history tree over it.	You can add records and produce an inclusion proof	CORE DATA STRUCTURE
2	Offline signing + anchoring. The hard one. Sign on device, chain locally, queue, submit hashes to two TSAs when connectivity returns.	You can state the bounded interval for any offline record	BACKDATING — THE MAKE-OR-BREAK
3	Hardware attestation. Key in Keystore/StrongBox; attestation certificate captured with Verified Boot state.	Server rejects a record whose attestation fails	DEVICE IDENTITY
4	Record 2 — the field test. Reagent, lot, expiry, operator, ambient, colour from a fixed vocabulary, reference table, presumptive-only flag.	A completed test record renders as a signed document	THE JOSHI ANSWER
5	Record 1 — the s.43 trigger. What the machine output, tier, time, place, device identity, what the officer did next.	Trigger and test records link to one event	THE NOVELTY CLAIM
6	Record 3 — the handoff. RPF-side transfer entry, GRP-side receipt, both signed, seal state and sample count carried across.	Two parties sign one transfer; verifier confirms both	THE SEAM
7	s.63 certificate generator. Auto-fill Schedule Part A and Part B fields, SHA-256, device make/model/serial.	Certificate exports as PDF from any record	COURT-FACING ARTEFACT
8	Verifier + tamper demo. CLI and web. Re-walk the chain, check anchors, pinpoint the broken link.	Editing a row in Postgres makes the verifier go red at that row	THE SHOWSTOPPER
9	Minimal dashboard. Case list, one case view, handoff status, certificate download. Nothing more.	A stranger can find a case and export its certificate	LEGIBILITY
10	Field note. Two weeks of real or simulated use. Log entry times, connectivity windows, what people refused to type.	You have numbers nobody else has	ORIGINALITY
11	Evidence pack + Q&A rehearsal. Claim-to-source table, do-not-claim list, hostile questions with sourced answers.	Every claim traces to a primary document	DEFENSIBILITY

WK	BUILD	DONE WHEN	RISK RETIRED
12	Buffer. Something will slip. Do not plan work here.	—	SCHEDULE RISK

GATE 1 — END OF WEEK 3

The spine works: a record signed on a disconnected phone, chained, anchored on reconnect, and verified. **If this is not working by week 3, cut records 1 and 3 and ship only the field-test record done perfectly.** One record that survives cross-examination beats three that do not.

GATE 2 — END OF WEEK 8

The tamper demo runs end to end, twice, on a machine that is not yours. If it only works on the developer's laptop, it does not work.

04 — THE DEMO

Six minutes, in this order

1. **Capture a trigger.** Phone in aeroplane mode. Record what the sensor said and what tier it produced. Show it signing offline.
2. **Run the field test.** Safe test substance, clearly labelled as such. Fill reagent, lot, expiry, observed colour from the dropdown. The record prints *presumptive — not a chemical identification* on its face.
3. **Reconnect.** Watch the queue drain to two independent timestamp authorities. Show the bounded interval: "created between 14:02 and 14:19."
4. **Hand off.** Second phone as GRP. Transfer signed by one, receipt signed by the other, sample count and seal state carried across.
5. **Export the certificate.** Section 63 Schedule, Part A and Part B, hash filled in, SHA-256 ticked. Say the sentence: *the statute requires a certificate in the prescribed form, and the prescribed form requires a hash.*
6. **Cheat.** Open Postgres. Edit one row. Re-run the verifier. It goes red and names the row.

THEN SAY THE LIMIT OUT LOUD

"This does not make the field test correct. It cannot. It makes the test's reagent, lot, operator and epistemic status legible to a court that today receives one sentence in a panchanama." Naming your own limit is what makes the rest of it credible.

05 – DO NOT BUILD

Four things to leave out, and what to say when asked

CUT Thermal imaging for concealment. Peer-reviewed rail-surveillance work says thermal cannot detect concealed objects; that needs millimetre-wave. On a 35 °C platform there is no skin-to-object contrast anyway.

CUT Behavioural anomaly CCTV. Four meta-analyses over 400+ studies: near chance. No video benchmark contains a concealed-contraband class, so you would have no labels and no baseline.

CUT Predictive patrol routing. The published trials ran at division scale on hundreds of thousands of incidents. One station cannot train it, and training on your own detections is a feedback loop.

CUT A three-node Hyperledger Fabric network. Its default ordering is crash-fault-tolerant, not Byzantine — 2 of 3 defeats it. India's own eSakshya chose hash-plus-immutable-storage, not a ledger.

When a judge asks why you dropped them: "We tested each against the evidence and three of them failed. Removing them is the finding." A team that subtracts with reasons reads as more competent than one that adds without them.

06 – STACK

Suggested, not prescribed

COMPONENT	SUGGESTION	WHY
Handheld	Android, Kotlin, Jetpack Compose	Hardware-backed Keystore and key attestation are the whole reason this works. iOS has equivalents but less accessible attestation.

COMPONENT	SUGGESTION	WHY
Keys	Android Keystore, StrongBox where present	Attestation certificate chains to Verified Boot state, signed by a key your app cannot reach.
Log store	Postgres, append-only by convention plus a trigger	Boring and inspectable. Being able to open it on stage is the demo.
Merkle tree	Any audited library, or ~200 lines yourself	Inclusion and consistency proofs. Writing it yourself is defensible at this scale and teaches you the proof.
Timestamps	RFC 3161 client, two independent authorities	Submits only a hash, never the record. Two authorities is what answers the single-trusted-party objection.
Backend	FastAPI or Node — whatever the team knows	Not where your marks are. Do not learn a new framework in week 4.
Certificate	Server-side PDF fill of the Schedule form	Part A and Part B are a fixed form. Auto-fill what you hold; leave the expert signature blank.

Companion to the phase-1 evidence review · run `narco-detection-evidence-chain-449fe5`

Every "cut" above traces to a finding in that report. Every week-2 risk is there because the research said so, not because it looked hard.